Faculty of Sciences Departement of Computer Science

Master thesis

Nauty (trouver un meilleur titre)

Auteur: Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2025–2026

Contents

Chapter 1

Preliminaries

This chapter introduces all the definitions necessary to understand this document. For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [2].

1.1 Algebra Definitions

Definition 1 (informal)**.** An *isomorphism* is a one-to-one correspondence (mapping) between two sets that preserves binary relationships between elements of the sets. If an element of one set is mapped to an element of the other set, this is denoted by $x \mapsto y$, meaning that the element x is mapped to the element y.

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has as identify element: 0, every element has an inverse element (each number with its opposite) and it's assoc iative: for all numbers a, b, c in $\mathbb{R}$: $a + (b + c) = (a + b) + c$.

A *homomorphism* is a transformation of one set into another that preserves in the second set the relations between elements of the first. For example, given two groups, $(G, \cdot)$ and $(H, \circ)$, a *group homomorphism* from $(G, \cdot)$ to $(H, \circ)$ is a function $\phi : G \to H$ such that $\phi(g_1 \cdot g_2) = \phi(g_1) \circ \phi(g_2)$ for every $g_1, g_2 \in G_1$.

Let $(G, *)$ be a group with identity element e, and let X be a non-empty set. A *group action* of G on X exists if there is a map

$$\cdot : G \times X \to X, \quad (g, x) \mapsto g \cdot x$$

such that the following two properties hold:

1. Identity: $\forall x \in X, \quad e \cdot x = x$,
2. Compatibility: $\forall g, h \in G,\ \forall x \in X, \quad (g * h) \cdot x = g \cdot (h \cdot x)$.

As an example, let S_n denote the *symmetric group* acting on a set X. We indicate the action of elements of S_n by exponentiation: for $x \in X$ and $g \in S_n$, x^g denotes the image of

x under g. The same notation is used to indicate the induced action on complex structures derived from X. As additional notation, for $W \subseteq V$, we write $W^g = \{w^g : w \in W\}$.

Definition 3. Let a group G act on a set X, and let $S \subseteq X$. The *pointwise stabilizer* of S in G is the subgroup defined by:

$$G_{(S)} = \{g \in G \mid \forall x \in S,\ g \cdot x = x\}.$$

Definition 4. Let $G \leq S_n$ be a permutation group and let $\alpha \in \{1, \dots, n\}$. The *orbit* of α under the action of G is the set

$$\mathrm{Orb}_G(\alpha) = \{\alpha^g \mid g \in G\}.$$

Definition 5. Let $G \leq S_n$ be a permutation group and let $\alpha \in \{1, \dots, n\}$. The *stabilizer* of α in G is the subgroup

$$G_\alpha = \{g \in G \mid \alpha^g = \alpha\}.$$

1.2 Graph Definitions

Definition 6. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V. The elements of V are the *vertices* of the graph G, the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the number of components of σ.If $\sigma = (v_1, v_2, \dots, v_k) \in V^*$ and $w \in V$ then $\sigma \mid\mid w$ denotes $(v_1, v_2, \dots, v_k, w)$. Furthermore, for $0 \leq s \leq k$,
$[\sigma]_s \coloneqq (v_1, v_2, \dots, v_s)$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*. Each vertex of a tree is called a *node*, and a node of degree 1 (only connected to one node) is called a *leaf*. When a particular node is designated as the starting point, that node is called the *root*. Given a tree T, a *subtree* T' is a connected subgraph of T that is itself a tree. We write $T' \subseteq T$ to denote that T' is a subtree of T.

Let $G = (V, E)$ be a graph. Let $\sim$ be an equivalence relation on V. The *quotient graph* of G with respect to $\sim$, denoted $G/\sim$, is a graph (V', E') such that:

1. $V' = V/{\sim} = \{[v] \mid v \in V\}$
2. $E' = \{[u][v] \mid uv \in E\}$

Definition 7. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \dots, n\}$).$\mathcal{G}$ represents the set of all graphs.

Definition 8. The *star graph* $K_{1,3}$ is a tree with one internal node and three leaves. It is also known as the *claw graph*. The internal node is called the *center* of the star graph. Pour la majorité de nos exemples, nous allons utiliser ce graphe.

1.2.1 Definitions specifics to isomorphisms

Definition 9. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \dots, k\}$ for some k. The number of colours, i.e. k, is denoted by $|\pi|$. A cell of π is

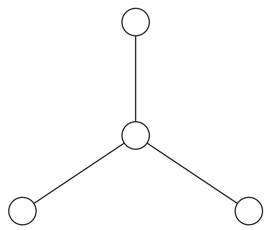

Figure 1.1: The star graph $K_{1,3}$ with its center and leaves.

the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. We let Π be the set of all colourings.

If π, π' are colourings, then π' is *finer than or equal* to π, written $\pi' \preccurlyeq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

Here, the colouring is not related to a *proper colouring*, π is related to a *partitioning*.

A *discrete colouring* is a colouring in which each cell is a singleton, in which case $|\pi| = n$.

A *coloured graph* is a pair (G, π) where π is a colouring of G.

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 10. Let S_n denote the *symmetric group* acting on V.

There are several notations to consider:

1. if π is a colouring of V, then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\mathrm{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}$; $\mathrm{Aut}(G, \pi)$ is the *Automorphism group* of coloured graph (G, π).

Definition 11. The *canonical form* is a function

$$C : \mathcal{G} \times \Pi \to \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

1.3 Complexity Definitions

For more informations on the complexity definitions, you can refer to Michael Sipser's book: Introduction to the Theory of Computation [7]

Definition 12 (minimalist)**.** The class NP (comes from nondeterministic polynomial) is the set of decision problems for which the problem instances have certificates verifiable in polynomial time by a deterministic Turing machine, or alternatively the set of problems that can be solved in polynomial time by a nondeterministic Turing machine.

A decision problem is a member of coNP iff its complement is in the complexity class NP.

A decision problem B is NP-*complete* if it satisfies the two following conditions:

1. B must belong to the class NP.
2. Every problem in NP can be reduced to B in polynomial time.

Definition 13. A problem is NP-*intermediate* (NPI) if it is not NP-complete nor P.

Definition 14. A language L belongs to the class *SPP* (**S**toic **PP**) if there exists a polynomial-time nondeterministic Turing machine M such that

$$x \in L \iff \#\mathrm{acc}_M(x) = 1 \text{ and } x \notin L \iff \#\mathrm{acc}_M(x) = 0,$$

where $\#\mathrm{acc}_M(x)$ denotes the number of accepting computation paths of M on input x.

Definition 15. $\mathsf{LOGSPACE}$ (or L) is the complexity class of decision problems that can be solved by a deterministic Turing machine using a logarithmic amount of writable memory.

Definition 16. An *oracle* for a language B is an external device that is capable of reporting whether any string w is a member of B. An oracle Turing machine is a modified Turing machine that has the additional capability of querying an oracle. We write M^B to describe an oracle Turing machine that has an oracle for language B.

The *Polynomial-Time Hierarchy* (PH) is a hierarchy of complexity classes that generalizes the classes NP and coNP. Formally, the base level of the hierarchy is defined as:

$$\Delta_0^{\mathsf{P}} \coloneqq \Sigma_0^{\mathsf{P}} \coloneqq \Pi_0^{\mathsf{P}} \coloneqq \mathsf{P},$$

where P denotes the class of decision problems solvable in polynomial time by a deterministic Turing machine. Then, for every $i \geq 0$, the hierarchy is defined recursively as follows:

- $\Delta_{i+1}^{\mathsf{P}} \coloneqq \mathsf{P}^{\Sigma_i^{\mathsf{P}}}$
- $\Sigma_{i+1}^{\mathsf{P}} \coloneqq \mathsf{NP}^{\Sigma_i^{\mathsf{P}}}$
- $\Pi_{i+1}^{\mathsf{P}} \coloneqq \mathsf{coNP}^{\Sigma_i^{\mathsf{P}}}$

Here, P^A denotes the class of problems solvable in polynomial time by a deterministic Turing machine equipped with an oracle for some complete problem in class A. The classes NP^A and coNP^A are defined analogously, using nondeterministic and co-nondeterministic Turing machines, respectively.

For example, $\Sigma_i^{\mathsf{P}} = \mathsf{NP}$, $\Pi_i^{\mathsf{P}} = \mathsf{coNP}$, and $\Delta_2^{\mathsf{P}} \coloneqq \mathsf{P}^{\mathsf{NP}}$ is the class of problems solvable in polynomial time with access to an oracle for an NP-complete problem.

This definition implies that each level of the hierarchy is contained within the next. That is:

- $\Sigma_i^{\mathsf{P}} \subseteq \Delta_{i+1}^{\mathsf{P}} \subseteq \Sigma_{i+1}^{\mathsf{P}}$

- $\Pi_i^{\mathsf{P}} \subseteq \Delta_{i+1}^{\mathsf{P}} \subseteq \Pi_{i+1}^{\mathsf{P}}$

The union of all classes in this hierarchy forms the polynomial-time hierarchy, denoted by PH.

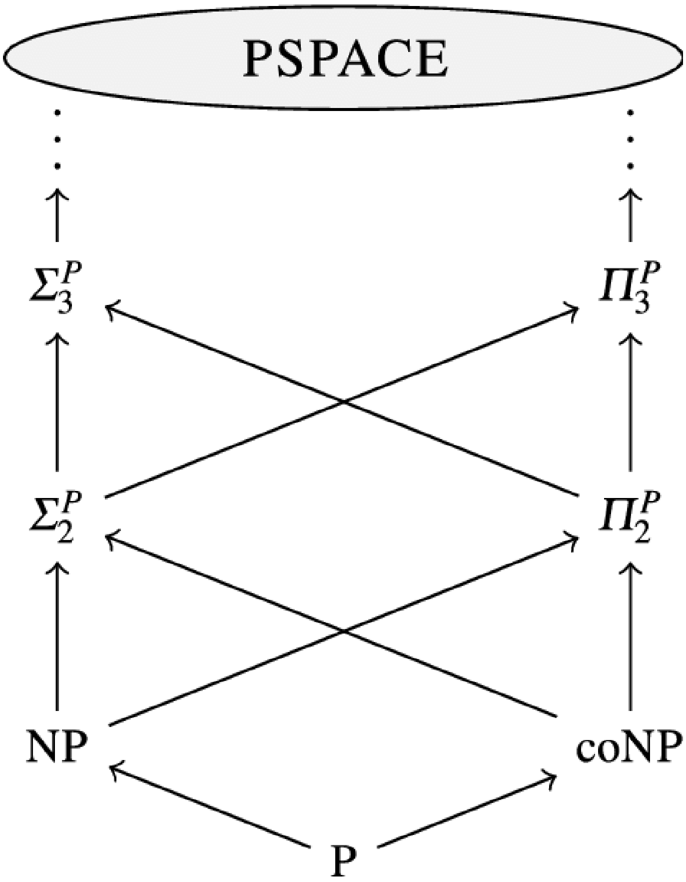

Figure 1.2: Polynomial Time Hierarchy where the arrows denote the inclusion: image from [1]

Chapter 2

Nauty Algorithm

In this section, we will focus extensively on McKay's general algorithm: Nauty.

2.1 Introduction of the algorithm

Nauty relies on the notion of a canonical form to achieve its main objective: finding a canonical labeling of a graph and, consequently, determining its automorphism group. The canonical form plays a central role in practice, as it constitutes a powerful tool for computing a canonical labeling.

The canonical form allows us to bypass the intrinsic difficulty of the graph isomorphism problem. Instead of directly comparing two graphs, we associate each graph with a unique and standardized representative, called its canonical graph (or canonical form). Two graphs are then isomorphic if and only if their canonical forms are identical. In this way, an equivalence problem is transformed into an equality problem, which is significantly easier to handle computationally.

We begin by introducing the algorithm itself and describing the different functions that compose it, as well as the process used to compute the canonical form of a graph. We then explain how the automorphism group can be constructed from this canonical form.

2.2 Search Tree construction

First of all, the algorithm itself is built upon a very specific type of tree: a search tree.

Informally, a search tree is a tree structure in which each node represents a state, and each branch corresponds to a specific action leading to another state. Since different actions produce different branches, the tree allows us to systematically explore all possible states starting from an initial state, with the goal of reaching a desired final state. In other words, we are performing a search over a tree structure, which explains the term *search tree*.

However, its particularity is that the leaves of this tree correspond to sequences of vertices of a coloured graph. As we move down the tree, these sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a "controlled" manner, the colouring of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete.The root of this tree is therefore the “empty” sequence.

To correctly construct this tree and find our canonical form, we will need several functions:

- *A refinement function*: Starting from a partition, this function allows us to construct a new partition that is finer than the previous one.
- *A target cell selection function*: Starting from a partition, this function allows us to choose a cell from this partition in order to create the “child” of the previous partition.
- *A node invariant function*: A function that allows us to efficiently compare the different nodes/leaves of our search tree in order to determine the canonical form of the graph.

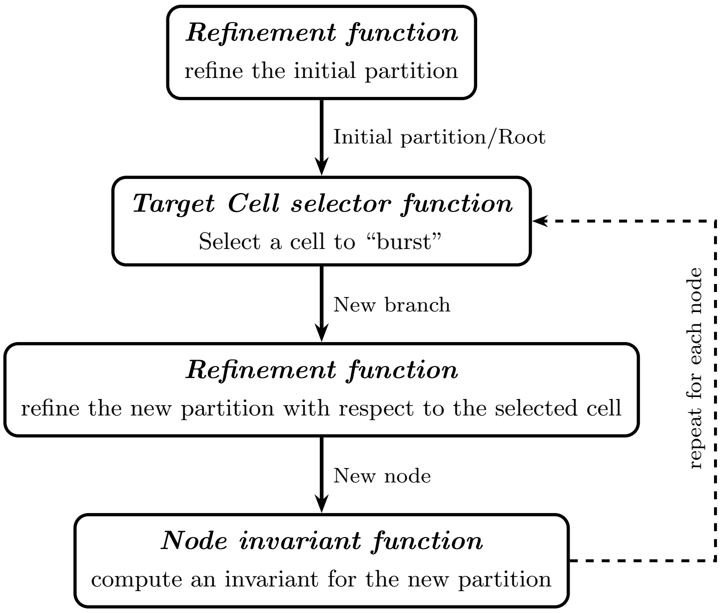

Figure 2.1: *search tree* construction in *Nauty*

These three functions are the fundamental components required to construct our search tree and find our canonical form. Before defining them formally, we will informally explain how they work and how they interact with each other to build our search tree.

Starting from a graph, we first apply the refinement function to obtain a so-called equitable partition (we will come back to this later when discussing the implementation of this function). This partition constitutes the root of our search tree.

Then, as long as we do not have a discrete partition, we apply the target cell selector to choose a cell from the current partition. Each branch represents a choice of element from the parent node’s selected cell. This cell is then split into two cells using an individualisation method (which will also be detailed later). For each branch, we apply the refinement function again to obtain a new partition, thereby creating a new node. Finally, on each node, we apply the node invariant function, which will be useful for determining the canonical form of our graph. We repeat this process until we obtain discrete partitions, which correspond to the leaves of our search tree. In summary, each node represents a partition, and each branch represents a cell chosen from the parent node to be individualised.

However, there exist additional functions that are more specific to the implementation of the algorithm and that will be explained in the implementation section of this document.

2.2.1 Formal definitions of the search tree functions

Now that we have introduced the different functions composing our algorithm, we can formally define them and establish the properties of the associated search tree.
Definition 17. The *Refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \to \Pi$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$,

R1. $R(G, \pi, \sigma) \preccurlyeq \pi$.

R2. if $\sigma_i \in \sigma$, then $\{\sigma_i\}$ is a cell of $R(G, \pi, \sigma)$.

R3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

In other words, the refinement function returns a colouring that is finer than π, and any vertex belonging to the sequence σ must form a singleton cell in the refined colouring. Furthermore, the function is equivariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before or after the refinement yields the same result.

Definition 18. The *Target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \to 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$

T1. if $R(G, \pi_0, \sigma)$ is discrete, then $T(G, \pi_0, \sigma) = \emptyset$.

T2. if $R(G, \pi_0, \sigma)$ is not discrete, then $T(G, \pi_0, \sigma)$ is a non-singleton cell of $R(G, \pi_0, \sigma)$.

T3. for any g $\in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

In other words, when the refinement function returns a discrete partition (corresponding to a leaf of the search tree) the target cell selector returns $\emptyset$. When the partition is non-discrete (corresponding to an internal node) it returns a non-singleton cell of that partition. As with the refinement function, the target cell selector is equivariant under permutation: applying any $g \in S_n$ to the inputs before or after the selection yields the same result.

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0). All the nodes of the tree are elements of V^*. This search tree has some characteristics to keep in mind:

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence ().
- if σ is a node of $\mathcal{T}(G, \pi_0)$, let W $= T(G, \pi_0, \sigma)$. Then the children of π are $\{\sigma \mid\mid w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

To demonstrate the correctness of the algorithm, the pruning procedure that we will introduce, and to prove that we indeed obtain the canonical form of the graph, we need to establish several properties of this search tree.

Lemma 1. *For any $G \in \mathcal{G}, \pi_0 \in \Pi, g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.*

Proof. Let $\sigma = (\sigma_1, \dots, \sigma_k)$ be a node of $\mathcal{T}(G, \pi_0)$. We first show that $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$. Such that $\sigma^g = (\sigma_1^g, \dots, \sigma_k^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

We proceed by induction on s, for $0 \leq s \leq k$, showing that $[\sigma^g]_s = (\sigma_1^g, \dots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Base case ($s = 0$): The empty sequence () is the root of any search tree. So $[\sigma^g]_0$ is trivially a node of $\mathcal{T}(G^g, \pi_0^g)$.

Inductive step: Assume $[\sigma^g]_s = (\sigma_1^g, \dots, \sigma_s^g)$ is a node of$\mathcal{T}(G^g, \pi_0^g)$.

Since $(\sigma_1, \dots, \sigma_s, \sigma_{s+1})$ is a node of $\mathcal{T}(G, \pi_0)$, the element σ_{s+1} was individualised to go from level s to level $s+1$. Since we apply g to the entire structure, σ_{s+1} is also mapped to σ_{s+1}^g, which is equivalent to directly individualising σ_{s+1}^g in G^g with π_0^g. Hence $[\sigma^g]_{s+1} = (\sigma_1^g, \dots, \sigma_s^g, \sigma_{s+1}^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

By induction, σ^g is a node of $\mathcal{T}(G^g, \pi_0^g)$, hence $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$.

The reverse inclusion follows by applying the same argument with g^{-1} in place of g, which gives $\mathcal{T}(G^g, \pi_0^g)^{g^{-1}} \subseteq \mathcal{T}(G, \pi_0)$, and therefore $\mathcal{T}(G^g, \pi_0^g) \subseteq \mathcal{T}(G, \pi_0)^g$. ☐

Corollary 2. *Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in Aut(G, \pi_0)$. Then σ^g, is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.*

Proof.
Part 1: σ^g is a node of $\mathcal{T}(G, \pi_0)$.
By Lemma 1, we have:

$$\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and therefore:

$$\mathcal{T}(G, \pi_0) = \mathcal{T}(G, \pi_0)^g$$

Since σ is a node of $\mathcal{T}(G, \pi_0)$, σ^g is a node of $\mathcal{T}(G, \pi_0)^g = \mathcal{T}(G, \pi_0)$.

Part 2: $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$
We apply Lemma 1 to the subtree rooted at σ. Since g maps each node of the subtree rooted at σ to a node of the subtree rooted at σ^g, and since $g \in \mathrm{Aut}(G, \pi_0)$ implies $G^g = G$ and $\pi_0^g = \pi_0$, the same argument as in Lemma 1 gives both inclusions:

$$\mathcal{T}(G, \pi_0, \sigma)^g \subseteq \mathcal{T}(G, \pi_0, \sigma^g) \quad \text{and} \quad \mathcal{T}(G, \pi_0, \sigma^g) \subseteq \mathcal{T}(G, \pi_0, \sigma)^g$$

and therefore $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$. ☐

Lemma 3. *Let σ be a node of (G, π_0) and let $\pi = R(G, \pi_0, \sigma)$. Then $\mathrm{Aut}(G, \pi)$ is the point-wise stabilizer of σ in $\mathrm{Aut}(G, \pi_0)$.*

Proof. We show both inclusions to establish the equality.

Part 1: $\mathrm{Aut}(G, \pi) \subseteq$ point-wise stabilizer of σ
Let $g \in \mathrm{Aut}(G, \pi)$. By R2, for each $\sigma_i \in \sigma$, $\{\sigma_i\}$ is a singleton cell of $\pi = R(G, \pi_0, \sigma)$. Since g is an automorphism of (G, π), it must map each cell of π to a cell of π. A singleton

$\{\sigma_i\}$ can only be mapped to itself, hence $\sigma_i^g = \sigma_i$ for every $\sigma_i \in \sigma$, so g stabilizes σ point-wise.

Part 2: point-wise stabilizer of $\sigma \subseteq \mathrm{Aut}(G, \pi)$
Let $g \in \mathrm{Aut}(G, \pi_0)$ be such that g stabilizes σ point-wise, that is $\sigma^g = \sigma$. We want to show that $g \in \mathrm{Aut}(G, \pi)$. So, $\pi^g = \pi$. By R3 applied to G, π_0 and σ:

$$R(G^g, \pi_0^g, \sigma^g) = R(G, \pi_0, \sigma)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and since g stabilizes σ point-wise, $\sigma^g = \sigma$. Therefore:

$$R(G, \pi_0, \sigma) = R(G, \pi_0, \sigma)^g$$

which gives exactly $\pi = \pi^g$, and hence $g \in \mathrm{Aut}(G, \pi)$. □

Now that we have properly defined our search tree, we can discuss the node invariant function and how it allows us to compute the canonical form of the graph.

Definition 19. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \to \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$.

The function ϕ has a few properties to take into account:

1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$,
   then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma_1' \in \mathcal{T}(G, \pi_0, \sigma')$ we have $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma_2)$.
2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete,
   then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^\pi = G^{\pi'}$ (equality relation).
3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.
4. Leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. This is the case, if there is a unique $g \in Aut(G, \pi_0)$ such that $\sigma^g = \sigma'$, with $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$.

Property 1 ensures that the ordering induced by ϕ is consistent across the search tree: if two sequences of equal length satisfy $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, then this strict inequality propagates to all descendant leaves, so no leaf below σ can exceed any leaf below σ'. Property 2 states that at the leaves (where both refined partitions are discrete) the invariant captures the canonical labelling exactly: two leaves yield equal invariants if and only if they produce the same canonically labelled graph. Property 3 expresses that ϕ is invariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before computing ϕ leaves the result unchanged.

Finally, Property 4 connects equivalence of leaves to the automorphism group. We recall that $R(G, \pi_0, \sigma)$ is the discrete partition obtained by applying the refinement function along the sequence σ, and can therefore be viewed as a permutation of the vertices. Since both leaves are equivalent, their node invariants are equal, which means there exists a permutation g such that:

$$R(G, \pi_0, \sigma)^g = R(G, \pi_0, \sigma')$$

Isolating g, we obtain:

$$g = R(G, \pi_0, \sigma)^{-1} \cdot R(G, \pi_0, \sigma')$$

If this permutation turns out to be an automorphism of (G, π_0), it means that both leaves describe the same graph up to a relabelling of vertices, and are therefore equivalent.

By Corollary 2, if σ is a leaf of $\mathcal{T}(G, \pi_0)$, then so is σ^g for any $g \in \mathrm{Aut}(G, \pi_0)$. Moreover, due to the properties of ϕ, all such leaves σ^g share the same ϕ-value, and no other leaf has this value. Consequently, for each leaf σ, we can say that:

$$\begin{aligned}\mathrm{Aut}(G, \pi_0) = \big\{R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1} : \sigma' \text{ is a leaf of } \mathcal{T}(G, \pi_0) \\ \text{with } \phi(G, \pi_0, \sigma') = \phi(G, \pi_0, \sigma)\big\}.\end{aligned}$$

We can define the canonical form:

$$\phi^*(G, \pi_0) \coloneqq \max \{\phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0)\},$$

and let σ^* be any leaf of $\mathcal{T}(G, \pi_0)$ that achieves the maximum. Now define $C(G, \pi_0) \coloneqq (G, \pi_0)^{R(G,\pi_0,\sigma^*)}$. By the properties of ϕ, $C(G, \pi_0)$ thus defined is independent of the choice of σ^*, which is consistent with the definition of the canonical form.

2.2.2 Implementation strategies

Now that we have all the theoretical tools in hand, we can begin discussing the implementation of the algorithm. Our implementation is based on the definitions of the algorithm, but it is not a direct or simplistic reproduction of its source code.

Implementation Refinement function

In order to ensure that the properties of the refinement function defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer) than the original colouring. This motivates the need for an algorithm that computes a coarsest equitable colouring, in order to prevent refinement from being too aggressive and skipping intermediate structure in the search tree.

To construct our refinement function, we need two auxiliary functions: a function that refines a colouring so that it becomes an equitable colouring, and an individualisation function.

Definition 20. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [5])

Data: π is the input colouring and α is a sequence of some cells of π
Result: the final value of π is the output colouring (equitable)
while α is not empty **and** π is not discrete **do**
  Remove some element W from α
  for each cell X of π **do**
    Let $X_1, \dots, X_k$ be the fragments of X distinguished according to the number of edges from each vertex to W
    Replace X by $X_1, \dots, X_k$ in π
    if $X \in \alpha$ **then**
      Replace X by $X_1, \dots, X_k$ in α
    else
      Add all but one of the largest of $X_1, \dots, X_k$ to α
    end if
  end for
end while

This function is, at first glance, relatively straightforward to understand. However, it is important to carefully examine the different steps involved in the refinement process. We therefore describe these steps in more detail below.

As stated in the pseudo-algorithm, we are given a graph G, a colouring π, and a sequence of cells of π that will be used to refine the colouring. As long as not all cells in the sequence have been processed and the colouring is not yet discrete, we continue refining the colouring.

At each iteration, we remove a cell W from the sequence. Then, for every cell X of the current colouring, we split X into several cells $X_1, \dots, X_k$ according to the number of edges each vertex of X has with the vertices of W (with respect to the graph G). We then replace the cell X in the colouring by the newly created cells $X_1, \dots, X_k$.

If the cell X already belongs to the sequence, we replace it in the sequence by the cells $X_1, \dots, X_k$. Otherwise, we add all cells among $X_1, \dots, X_k$ to the sequence except one, namely a largest cell. If several cells have the same maximum size, one of them is chosen arbitrarily and is not added to the sequence.

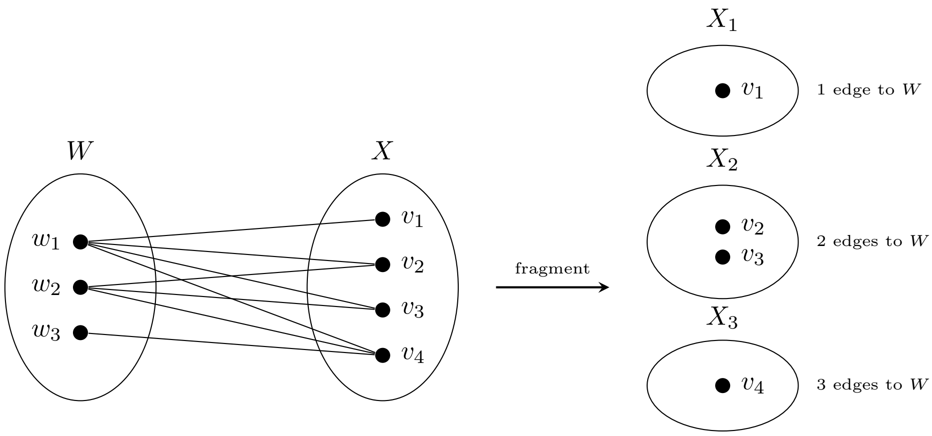

Figure 2.2: Fragmentation of cell X with respect to W

Definition 21 (Individualization)**.** Let $\pi \in \Pi$ and let $v \in V$ be a vertex belonging to a non-singleton cell of π. The *individualization function*

$$I : \Pi \times V \to \Pi$$

maps (π, v) to the partition $\pi' = I(\pi, v)$ defined, for every $w \in V$, by

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v, \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

The vertex v is said to be *individualized* in π. Equivalently, $I(\pi, v)$ is obtained from π by assigning a unique colour to the vertex v.

The purpose of this function is to construct a new partition from a given partition by individualizing a vertex. More precisely, the operation assigns a unique colour to a specific vertex v by removing it from its original cell and placing it into a new singleton cell. This new cell is inserted immediately before the original cell containing v.

Now we can define our refinement function. For a sequence of vertices v_1,v_2,. . ., define

- $R(G, \pi_0, ()) = F(G, \pi_0,\ \text{a list of all the cells of } \pi_0)$
- $R(G, \pi_0, (v_1)) = F(G, I((R(G, \pi_0, ()), v_1), \{v_1\}))$
- $R(G, \pi_0, (v_1, v_2)) = F(G, I((R(G, \pi_0, (v_1)), v_2), \{v_2\}))$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G, I((R(G, \pi_0, (v_1, v_2)), v_3), \{v_3\}))$
- and so on.

The algorithm satisfies the required properties of a refinement function and performs well in practice. However, the refinement step is the most computationally expensive part of the algorithm, which is why special care must be taken in its implementation. C'est pourquoi, pour réduire les temps de calculs, nous allons utiliser des méthodes de prunning dont nous parlerons un peu plus tard.

Implementation Target Cell Selector

For the implementation of the target cell selector function in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilizes the current sequence [4]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [3].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

In our implementation we will only use the first strategy.

pseudo-code of targetCellSelector

1: **Data:** a partition $\pi = (X_1, \ldots, X_k)$
2: **Result:** the first non-singleton cell of minimum size in π
3: **return** the first $X \in \pi$ such that $|X| > 1$ and $|X| = \min_{Y \in \pi,\ |Y|>1} |Y|$

Implementation Node Invariant

Now that the search tree is implemented, we need to define the node invariant function, which will allow us to determine the canonical form. To compute this function, we can rely on two related sources:

1. For each node σ, there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.
2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \ldots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph* $Q(G, \pi)$ (see Definition 6), assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π, and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possibly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2. The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [v]_i))$ for $i = 0, \ldots, |v|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

In our implementation, we will use the first source described above. We rely on the number and size of the cells, and we apply combinatorial properties of the coloured graph to compute our node invariant. Several invariant methods have been implemented in order to test their efficiency and computational speed.

Since this function is called at each node creation, it is important that its computation remains fast in order to avoid slowing down the algorithm. The following sections detail each of these methods, along with a brief discussion of their computational complexity and how they compare to one another.

Here is the exhaustive list of invariant methods that we have implemented so far:

1. *Number of triangles per cell*: for each cell of the partition, we compute the number of triangles contained in that cell. We then obtain a vector where each entry corresponds to the number of triangles in the corresponding cell. Finally, we apply a lexicographic ordering to this vector to compute our node invariant.

For counting the number of triangles in a cell, the most classical approach enumerates all triplets of vertices (u, v, w) from the cell and checks for each one whether all three edges exist, running in $O(n^3)$ where n is the number of vertices in the cell. A faster approach exploits the sorted neighbor lists $N(v)$: for each vertex $u \in C$, one iterates over its neighbors v in the cell with $v > u$, then looks for common neighbors w of both u and v still in the cell with $w > v$. Since the neighbor lists are sorted, each such lookup runs in $O(d)$ where d is the largest number of neighbors that any vertex in the cell has, yielding an overall complexity of $O(n \cdot d^2)$.

Pseudo-code: Number of triangles per cell

Data: A graph G, a cell $C = \{v_1, \ldots, v_k\}$
    $N(u)$ denotes the ordered neighbor list of u in G
Result: The number of triangles contained in C

$count \leftarrow 0$
for each $u \in C$ **do**
    for each $v \in N(u) \cap C$ such that $v > u$ **do**
        for each $w \in N(u) \cap N(v) \cap C$ such that $w > v$ **do**
            $count \leftarrow count + 1$
        end for
    end for
end for
return *count*

[**Rodolphe:** les prochaines méthodes d'invariants arrivent mais la structure est là]

2.2.3 Pruning strategies

To avoid generating unnecessary information in our search tree,which would result in an overly large tree, we aim to construct the most optimized tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define two types of pruning strategies to optimize our construction:

$P_A(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

$P_B(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

$P_C(\sigma, g)$: Suppose $g \in \mathrm{Aut}(G, \pi_0)$ and suppose $\sigma < \sigma'$ are nodes of $T(G, \pi_0)$ such that $\sigma^g = \sigma'$. Then, we will remove $T(G, \pi_0, \sigma')$.

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.
Theorem 4. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $P_A(\sigma, \sigma')$ or $P_C(\sigma, g)$ are performed. Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.*
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $P_B(\sigma, \sigma')$ $P_C(\sigma'', g)$ are performed,*

where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$*. Let* $g_1, \ldots, g_k$ *be the automorphisms used in the operations* P_C *that were performed, and let* $A = \{g \in \mathrm{Aut}(G, \pi_0)\colon v_0^g$ *is a remaining leaf*$\}$*. Then* $\mathrm{Aut}(G, \pi_0)$ *is generated by* $\{g_1, \ldots, g_k\} \cup A$*.*

[**Rodolphe:** Faire preuve]

The P_A pruning strategy is relatively straightforward to implement. At each level of the search tree, we simply compare the node invariant of the node currently being explored with the best node invariant encountered so far at the same level. If the invariant of the current node is smaller than the best one found previously, then the corresponding subtree can safely be discarded.

The P_B strategy is slightly more involved. Instead of comparing the current node with the best node encountered so far, we compare its node invariant with that of a reference node. This reference node is chosen as the node at the same level lying on the first path leading to the first leaf. If the two node invariants differ, the current subtree is pruned. Conversely, branches whose node invariants coincide with those of the reference path remain candidates for discovering new automorphisms.

Finally, the P_C strategy exploits the automorphisms that have already been discovered during the search. Suppose that an automorphism $g \in \mathrm{Aut}(G, \pi_0)$ maps the current node σ to another node σ^g that has already been explored. Since the corresponding subtrees are isomorphic, exploring both of them would be redundant. Consequently, the subtree rooted at σ^g can be discarded.

Implementing this strategy is considerably more challenging than implementing P_A or P_B, and several approaches have been proposed over the years. In the earliest versions of *Nauty*, pruning relied solely on the automorphisms discovered during the search. More recent versions additionally provide an optional *Random Schreier* method, which *Nauty* adopted in place of a deterministic alternative for its greater efficiency, and which constructs a randomized approximation of the automorphism group from the automorphisms already found.This often enables more aggressive pruning, although its effectiveness depends on the input graph; for some graph families it provides little or no improvement and may even introduce additional overhead.

In this work, we implement the original approach, which relies only on the automorphisms discovered during the search. To represent and manipulate the automorphism group efficiently, we use the *Schreier-Sims algorithm*, which constructs a base and strong generating set (BSGS) together with the associated stabilizer chain, allowing efficient orbit computations and pruning decisions.

Schreier-Sims Algorithm

A permutation group $G \leq S_n$ given by a set of generators can be extremely large, growing combinatorially with n, so working with it by listing its elements one by one is simply not an option. The Schreier-Sims algorithm avoids this by building a compact representation of G, from which the order $|G|$, membership testing, and orbit computations can all be obtained efficiently.

The idea is to shrink the group step by step. Starting from G, we choose a point b_1 and restrict our attention to the permutations of G that happen to fix it: this smaller group is the *stabilizer* of b_1. We then choose a second point b_2 and repeat the same reduction inside this smaller group, and so on, until only the identity permutation is left. The sequence of chosen points $b_1, \ldots, b_k$ is called a *base*.

This reduction is made computationally cheap by two related notions. The *orbit* of the current base point is the set of points it can be mapped to by the current group; it can be computed by a breadth-first search using only the group's generators. This same search yields a *transversal*, that is, for every point in the orbit, a permutation of the current group mapping the base point to it. Combining the transversal with the current generators, through the so-called *Schreier generators*, produces a generating set for the next, smaller stabilizer, without ever enumerating the elements of the group itself.

This process gives a base together with a generating set for each successive stabilizer. It is called a *Base and Strong Generating Set* (BSGS). It also gives an immediate way to compute the order of G: by the orbit-stabilizer theorem, $|G|$ is simply the product of the orbit sizes obtained at each step,

$$|G| = \prod_{i=1}^{k} o_i,$$

where o_i is the size of the orbit of b_i computed at step i.

To better understand the construction, we consider the graph $K_{1,3}$. We build the BSGS associated with $\mathrm{Aut}(K_{1,3})$ using the base $(b_1 = 1, b_2 = 2)$.

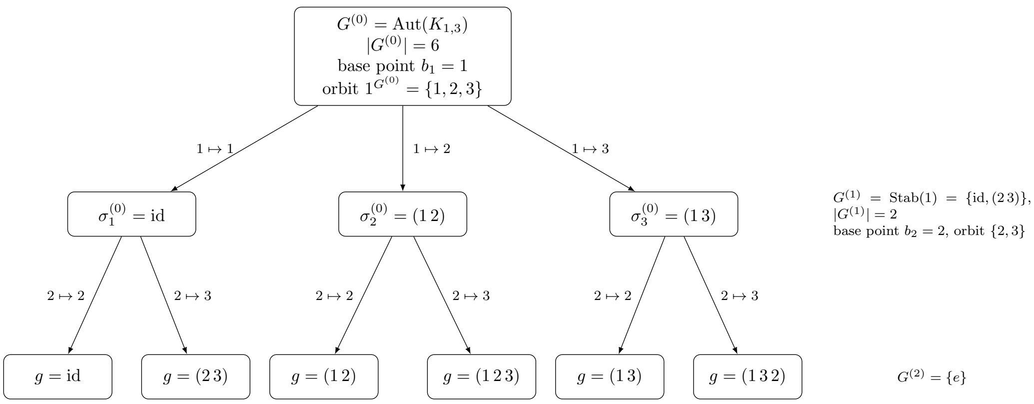

Figure 2.3: Tree associated with the BSGS of $\mathrm{Aut}(K_{1,3})$ with base $(b_1 = 1, b_2 = 2)$.

We observe that each edge at level i is labelled by an element $\sigma_p^{(i)}$ of the transversal associated with the orbit of b_{i+1} under $G^{(i)}$. Each leaf corresponds to an element $g \in G$, obtained by composing the transversal representatives along the path from the root. The $3 \times 2 \times 1 = 6$ leaves therefore represent exactly the six elements of $\mathrm{Aut}(K_{1,3})$.

Schreier-Sims for *Nauty*

Let G be the input graph and π_0 its initial partition.

Nauty performs a depth-first traversal of the search tree, and at each leaf it is possible to discover a new automorphism of the graph and add it to the urrently known automorphism group. Whenever two distinct leaves σ_0 (the first leaf) and σ' produce the same invariant, the permutation $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$ is an automorphism of G.

These automorphisms, discovered one by one during the depth-first traversal, form the generators from which the group structure is built. It is therefore not necessary to know

$\mathrm{Aut}(G, \pi_0)$ in advance: the generators emerge dynamically during the search, and the group structure is updated incrementally as new automorphisms are discovered.

The automorphism g is then sifted through the existing stabilizer chain, correcting its action on each base point successively. It is either absorbed by an existing level, in which case it provides no new information, or it is identified as a new strong generator and enriches the chain.

Each automorphism discovered at a leaf is propagated to the ancestor nodes for which it remains relevant. The accumulated group information is then used when selecting the next vertex to individualize. Once a target cell has been chosen, its vertices are partitioned into orbits under the currently known automorphism group. Two vertices belonging to the same orbit necessarily lead to isomorphic subtrees, since they are related by an automorphism already known to the algorithm. Consequently, it is sufficient to explore a single representative of each orbit, while all other vertices can be pruned without exploration.

This is a direct application of the pruning operation P_C.

To get a concrete idea of what this looks like in code, here is a simplified pseudo-code of the Schreier-Sims method for *Nauty*.

pseudo-code of Schreier-Sims within Nauty

Data: reference leaf σ_0 (the first leaf reached), current leaf σ', group data (B, Δ_i, U_i) built so far
Result: group data updated with any automorphism found at σ'
if invariant(σ') = invariant(σ_0) **then**
    $g \leftarrow R(G, \pi_0, \sigma')\, R(G, \pi_0, \sigma_0)^{-1}$ ▷ candidate automorphism
    if $g \neq \mathrm{id}$ **then**
        InsertGenerator(g, level 1) ▷ see previous section
    end if
end if

Random Schreier method

The main difference between the classical Schreier-Sims algorithm seen previously and the Random Schreier method lies in the way the BSGS is constructed. Instead of deterministically maintaining an exact base and strong generating set from every newly discovered automorphism, the Random Schreier method adopts a probabilistic approach. It repeatedly generates random products of the automorphisms already known and inserts these permutations into the Schreier-Sims structure.

The objective is not to compute the complete BSGS as early as possible, but rather to obtain a sufficiently rich approximation of the subgroup generated by the discovered automorphisms. This approximation often provides accurate orbit information much earlier during the search, allowing the P_C pruning strategy to eliminate redundant branches sooner. Consequently, the search tree may become significantly smaller.

Since the method relies on randomization, its effectiveness depends on the input graph. For many highly symmetric graphs, it substantially improves the pruning process. However, for graphs with few automorphisms, the additional computations required to generate and process random group elements may outweigh the benefits. For this reason, the Random Schreier method is implemented as an optional feature in *Nauty*.

2.2.4 Final Search Tree Construction

We can put in practice ours different pruning strategies on our search tree. *Nauty* will generate its tree in depth-first order. The lexicographically least leaf σ_1 is kept and if the canonical labelling is sought (rather than just the automorphism group), the leaf σ^* with the greatest invariant discovered so far is also kept. A non-discrete node σ is pruned if neither $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, [\sigma_1]_{|\sigma|})$ or $\phi(G, \pi_0, \sigma) \geq \phi(G, \pi_0, [\sigma^*]_{|\sigma|})$. Such operations have both type $P_A(\sigma^*, \sigma)$ and $P_B(\sigma_1*, \sigma)$. We can therefore conclude that only a subset of the tree will be generated, and that the generated leaves are those that may be "better" than the leaves already discovered.

Pseudo-code of canonical labelling

Data: A graph $G = (V, E)$
Result: The canonical form of G
$\pi \leftarrow (\{0, 1, \ldots, |V| - 1\})$
$\pi \leftarrow F(G, \pi,$ a list of all the cell of $\pi)$
$\pi^* \leftarrow \pi$ ▷ π^* the best partition
$\pi_1 \leftarrow ()$ ▷ π_1 the partition of first leaf
$\sigma \leftarrow (), \quad \sigma^* \leftarrow ()$ ▷ current and best vertex sequences
procedure NAUTY(G, π)
    if π is discrete **then**
        if π^* is not discrete **then** ▷ no leaf found yet
            $\pi^* \leftarrow \pi$, $\sigma^* \leftarrow \sigma$, $\pi_1 \leftarrow \pi$
            BSGS $\leftarrow$ Base (σ) ▷ initialize Base
        else if $\phi(G, \pi, \sigma) > \phi(G, \pi^*, \sigma^*)$ **then**
            $\pi^* \leftarrow \pi$, $\sigma^* \leftarrow \sigma$ ▷ better leaf found
        end if
        return
    end if
    if π^* is discrete **and** $\phi(G, \pi, \sigma) < \phi(G, \pi^*, \sigma^*)$ **and** $\phi(G, \pi, \sigma) \neq \phi(G, \pi_1, \sigma_1)$ **then**
        return ▷ Pruning $P_A + P_B$: cannot beat π^*, nor tie π_1
    end if
    Cell $\leftarrow T(G, \pi, \sigma)$
    $R \leftarrow$ orbit representatives of Cell ▷ Pruning P_C
    for each $\nu \in R$ **do**
        $\pi_\nu \leftarrow I(\pi, \nu)$
        $\pi_\nu \leftarrow F(G,\ \pi_\nu,\ \{\nu\})$
        Append ν to σ
        NAUTY(G, π_ν)
        Remove last entry from σ
    end for
end procedure
$g \leftarrow$ canonical vertex ordering induced by π^*
return G^g ▷ Apply g to G to get the canonical form

The graph encoding is not fixed; however, we adopt the same approach as nauty, namely representing input graphs using the graph6 format [6]. This choice enables more efficient testing of our algorithm.

Bibliography

[1] S. Baruah. Feasibility analysis for hpc-dag tasks. *Real-Time Systems*, 58:1–19, June 2022. DOI: 10.1007/s11241-022-09380-z.

[2] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.

[3] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.

[4] B. D. McKay. Practical graph isomorphism, 1981.

[5] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.

[6] B. D. McKay. Description of graph6, sparse6 and digraph6 encodings. https://users.cecs.anu.edu.au/~bdm/data/formats.txt, 2022.

[7] M. Sipser. Introduction to the theory of computation. *ACM Sigact News*, 27(1):27–29, 1996.